BDY1103 – TALLER DE BASE DE DATOS

Evaluación Parcial N° 2 – Caso de Análisis Semestral

Sistema de Gestión de Base de Datos para una Clínica

Situación evaluativa 1: Entrega de encargo (informe)

Estudiante: Sebastian Antipan

DUOC UC – Escuela de Informática y Telecomunicaciones

Introducción

Descripción del Proyecto

Este informe corresponde a la segunda parte del caso semestral, Sistema de Gestión de Base de Datos para una Clínica. En la Evaluación Parcial 1 se construyó el modelo de datos completo (12 tablas relacionadas) y se cargó un volumen masivo de información de prueba (120 pacientes y, en cascada, citas, diagnósticos, historiales, tratamientos, recetas y receta-detalle), además de bloques PL/SQL fundamentales (RECORD, VARRAY, cursores, excepciones y un primer acercamiento a procedimientos, funciones, paquetes y triggers).

En esta segunda etapa, el objetivo es completar y formalizar la capa procedural de la base de datos: se implementan procedimientos almacenados con y sin parámetros, funciones almacenadas con y sin parámetros, un paquete (package) con parte pública y parte privada, y triggers tanto a nivel de fila como a nivel de sentencia. Distintos objetos PL/SQL se utilizan para cumplir el objetivo del proyecto: automatizar el registro de citas, generar reportes masivos a partir de los 120 pacientes, validar datos antes de insertarlos y registrar una bitácora de accesos a la información, de manera que la solución pueda ser reutilizada por otros programas PL/SQL y por sentencias SQL.

Contexto de negocio y datos a procesar

La clínica atiende consultas de medicina general y especialidades. Su operación diaria gira en torno a cuatro procesos: agendar una cita, atender al paciente y registrar el diagnóstico y el tratamiento, emitir recetas que descuentan stock de medicamentos, y generar reportes de gestión. Cuando cada programa repite esas validaciones y consultas por su cuenta, aparecen citas con pacientes o médicos inexistentes, stock desactualizado, reportes inconsistentes y falta de trazabilidad. La EP2 ataca ese problema llevando las reglas y los cálculos al interior de la base de datos.

Los datos que se procesan son las 12 tablas del modelo: paciente (120 registros), medico (10), consultorio (6), especialidad (10), medicamento (12), cita (241 generadas en la carga masiva, más 2 de las pruebas de la EP1), diagnostico (241), historial_medico (120), tratamiento (165), receta (80), receta_detalle (113) y medico_especialidad (12). La tabla bitacora_citas se agrega en la EP2 como apoyo del trigger de sentencia.

La información relevante que se genera es: el resumen general de la clínica (pacientes, médicos, citas y stock), la edad de cada paciente, la cantidad de citas por médico, la validación de una cita antes de insertarla y la bitácora de las inserciones en la agenda. Así, la base no solo almacena información: también la procesa y entrega resultados listos para la gestión.

Alcance

El alcance de esta entrega cubre la implementación y demostración de:

- Un procedimiento con parámetros (sp_registrar_cita) que valida la existencia de paciente y médico antes de insertar una cita.
- Un procedimiento sin parámetros (sp_resumen_clinica) que procesa información masiva generando el resumen general de la clínica.
- Una función con parámetros (fn_calcular_edad), reutilizada dentro de sentencias SQL y de otros programas PL/SQL.
- Una función sin parámetros (fn_total_pacientes), invocable desde sentencias SQL y desde otros bloques PL/SQL.
- Un paquete pkg_reportes_clinica con elementos públicos (declarados en la especificación) y elementos privados (declarados solo en el cuerpo), que integra validación y reportes en una sola solución.
- Un trigger a nivel de fila (trg_descontar_stock, Parcial 1) y un trigger a nivel de sentencia (trg_bitacora_cita), que junto a la tabla de apoyo bitacora_citas entregan trazabilidad y automatización.

Quedan fuera del alcance de esta entrega la interfaz gráfica de usuario, la integración con sistemas externos de facturación o previsional, y el desempeño sobre volúmenes de datos superiores a los generados para el caso (los cuales serían tema de una siguiente etapa con técnicas como BULK COLLECT o particionamiento).

Tecnologías Utilizadas

- Oracle Database (motor de base de datos relacional).
- PL/SQL (lenguaje procedural de Oracle para bloques, procedimientos, funciones, paquetes y triggers).
- Oracle SQL Developer (entorno de desarrollo y ejecución de scripts).
- DBMS_OUTPUT para la salida de mensajes y resultados de prueba en consola.

Desarrollo de procedimientos y funciones

Definición y características

Un procedimiento almacenado es un bloque PL/SQL con nombre que se guarda compilado dentro de la base de datos y que ejecuta una acción: insertar, actualizar, borrar, validar o generar reportes. Puede recibir parámetros de entrada (IN) y declararse con o sin ellos; se invoca por su nombre desde otro programa PL/SQL o desde una sentencia SQL.

Una función almacenada es similar, pero su objetivo es CALCULAR y DEVOLVER exactamente un valor mediante RETURN. Por esa razón puede usarse directamente dentro de sentencias SQL (SELECT, WHERE o expresiones), algo que un procedimiento —que no devuelve valor— no puede hacer.

Ventajas y desventajas de cada uno

Procedimientos almacenados —Ventajas:

- La lógica queda guardada y compilada en la base: se reutiliza sin reescribirla y todos los programas la usan igual.
- Pueden ejecutar varias operaciones y controlar su propio manejo de errores (EXCEPTION).
- Con parámetros se adaptan a distintos datos de entrada sin cambiar el código (sp_registrar_cita sirve para cualquier paciente y médico).

Procedimientos almacenados —Desventajas:

- No devuelven valores con RETURN: para devolver resultados hay que imprimirlos con DBMS_OUTPUT o dejarlos en tablas.
- Al vivir en la base, un error de lógica queda centralizado: si el procedimiento falla, falla para todos los programas que lo usan.

Funciones almacenadas —Ventajas:

- Devuelven un valor y por eso se integran naturalmente en sentencias SQL, filtros y expresiones.
- Permiten encapsular cálculos reutilizables (por ejemplo, la edad de un paciente se calcula igual en todas las consultas).

Funciones almacenadas —Desventajas:

- Deben tener un RETURN obligatorio y retornar en todos los caminos posibles (incluyendo las excepciones).
- Cuando se usan dentro de sentencias SQL sobre tablas grandes, una función que consulta datos por cada fila puede afectar el rendimiento.

Propósito de los procedimientos y funciones en el proyecto

En la clínica los procedimientos cubren las acciones del negocio y las funciones cubren los cálculos que se reutilizan:

- sp_registrar_cita (con parámetros): registra una cita validando que el paciente y el médico existan, evitando citas con datos inválidos.
- sp_resumen_clinica (sin parámetros): procesa la información masiva (120 pacientes, 10 médicos, todas las citas y el stock de medicamentos) y genera el resumen general de la clínica sin necesitar ningún dato de entrada.
- fn_calcular_edad (con parámetros): calcula la edad de un paciente a partir de su fecha de nacimiento y puede usarse dentro de un SELECT para presentarla junto a cada paciente.
- fn_total_pacientes (sin parámetros): entrega el total de pacientes atendidos por la clínica; funciona sin entrada porque opera sobre todo el conjunto de datos.

Demostración: procedimiento sin parámetros

El procedimiento sp_resumen_clinica calcula los totales de las tablas principales y los imprime por pantalla. No recibe parámetros porque el reporte abarca siempre a toda la información cargada:

```
CREATE OR REPLACE PROCEDURE sp_resumen_clinica IS
    v_total_pacientes    NUMBER;
    v_total_medicos      NUMBER;
    v_total_citas        NUMBER;
    v_stock_medicamentos NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total_pacientes FROM paciente;
    SELECT COUNT(*) INTO v_total_medicos   FROM medico;
    SELECT COUNT(*) INTO v_total_citas     FROM cita;
    SELECT NVL(SUM(stock), 0) INTO v_stock_medicamentos FROM medicamento;

    DBMS_OUTPUT.PUT_LINE('===== RESUMEN GENERAL DE LA CLINICA =====');
    DBMS_OUTPUT.PUT_LINE('Pacientes registrados    : ' || v_total_pacientes);
    DBMS_OUTPUT.PUT_LINE('Medicos registrados      : ' || v_total_medicos);
    DBMS_OUTPUT.PUT_LINE('Citas registradas        : ' || v_total_citas);
    DBMS_OUTPUT.PUT_LINE('Unidades de medicamentos : ' || v_stock_medicamentos);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error inesperado: ' || SQLERRM);
END sp_resumen_clinica;
/
```

Uso en un programa PL/SQL y en sentencias SQL. El mismo procedimiento se invoca directamente desde SQL y a la vez se reutiliza dentro de otro bloque PL/SQL sin copiar su lógica:

```
-- Validacion 1: invocacion del procedimiento desde una sentencia SQL.
CALL sp_resumen_clinica();

-- Validacion 2: invocacion del procedimiento desde un bloque PL/SQL.
DECLARE
    v_mensaje VARCHAR2(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Ejecutando el reporte general automaticamente...');
    sp_resumen_clinica;
    v_mensaje := 'Reporte generado el ' || TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI');
```

```
    DBMS_OUTPUT.PUT_LINE(v_mensaje);
END;
/
```

Demostración: procedimiento con parámetros

El procedimiento sp_registrar_cita (desarrollado en el Parcial 1 y mantenido en esta etapa) recibe los datos de la cita como parámetros y valida la existencia del paciente y del médico antes de insertar:

```
CREATE OR REPLACE PROCEDURE sp_registrar_cita (
    p_id_paciente     IN paciente.id_paciente%TYPE,
    p_id_medico       IN medico.id_medico%TYPE,
    p_id_consultorio  IN consultorio.id_consultorio%TYPE,
    p_fecha_cita      IN DATE,
    p_hora_cita       IN VARCHAR2
) IS
    v_existe_paciente NUMBER;
    v_existe_medico   NUMBER;
    v_nuevo_id_cita   NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_existe_paciente
    FROM paciente WHERE id_paciente = p_id_paciente;

    SELECT COUNT(*) INTO v_existe_medico
    FROM medico WHERE id_medico = p_id_medico;

    IF v_existe_paciente = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No se puede registrar la cita: el paciente '
            || p_id_paciente || ' no existe.');
        RETURN;
    END IF;

    IF v_existe_medico = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No se puede registrar la cita: el medico '
            || p_id_medico || ' no existe.');
        RETURN;
    END IF;

    SELECT NVL(MAX(id_cita), 0) + 1 INTO v_nuevo_id_cita FROM cita;

    INSERT INTO cita (
        id_cita, id_paciente, id_medico, id_consultorio,
        fecha_cita, hora_cita, estado
    ) VALUES (
        v_nuevo_id_cita, p_id_paciente, p_id_medico, p_id_consultorio,
        p_fecha_cita, p_hora_cita, 'PROGRAMADA'
    );
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Cita #' || v_nuevo_id_cita || ' registrada correctamente.');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error al registrar la cita: ' || SQLERRM);
END sp_registrar_cita;
/
```

Su llamada desde un bloque PL/SQL así lo demuestra (paciente 1, médico 2, consultorio 3, dentro de una semana a las 10:30), y el resultado 'Cita #242 registrada correctamente' confirma tanto la validación previa como la inserción.

Demostración: funciones con y sin parámetros

Por una parte, fn_calcular_edad (Parcial 1) recibe el id del paciente y devuelve su edad calculada. Por otra, fn_total_pacientes (Parcial 2) no recibe nada y devuelve el total de pacientes. Ambas se muestran usadas en sentencias SQL y en otros programas PL/SQL:

```
CREATE OR REPLACE FUNCTION fn_total_pacientes RETURN NUMBER IS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM paciente;
    RETURN v_total;
EXCEPTION
    WHEN OTHERS THEN
        RETURN -1;
END fn_total_pacientes;
/

-- Validacion 1: uso de la funcion en una sentencia SQL.
SELECT fn_total_pacientes AS total_pacientes_de_la_clinica FROM dual;

-- Validacion 2: uso de la funcion en un bloque PL/SQL.
DECLARE
    v_total NUMBER;
BEGIN
    v_total := fn_total_pacientes;
    DBMS_OUTPUT.PUT_LINE('La clinica atiende hoy a ' || v_total || ' pacientes.');
END;
/

-- Uso de fn_calcular_edad en SQL sobre 10 pacientes.
SELECT id_paciente,
       nombre || ' ' || apellido AS nombre_completo,
       fn_calcular_edad(id_paciente) AS edad
FROM paciente
WHERE ROWNUM <= 10
ORDER BY id_paciente;
```

La consulta anterior devuelve, para cada uno de los primeros 10 pacientes, su nombre completo y la edad calculada por la función, evidenciando el uso masivo de la función dentro de una sentencia SQL.

Desarrollo de packages

Definición y características

Un package (paquete) agrupa procedimientos, funciones y variables relacionados bajo un mismo nombre lógico, y se compone de dos bloques:

- Especificación (spec): la parte PÚBLICA. Declara qué elementos existen y pueden ser invocados por cualquier programa o sentencia SQL.
- Cuerpo (body): implementa la lógica y admite elementos PRIVADOS (variables y funciones que no aparecen en la spec). Esos elementos solo son visibles para el propio package, funcionando como helpers internos o 'constructores' de datos internos.

Ventajas y desventajas

Ventajas:

- Organización: la lógica de un módulo completo del negocio (por ejemplo, los reportes de pacientes y médicos) queda agrupada en un solo objeto con nombre pkg_*.
- Encapsulamiento: la parte privada esconde detalles internos (una variable de control o una función validadora) que otros programas no deben tocar.

- Mantenimiento: si cambia la implementación interna, la especificación pública no cambia, y los programas que usan el package no se rompen.
- Las variables del package mantienen su valor durante toda la sesión, lo que permite compartir contadores o configuraciones entre llamadas.

Desventajas:

- Obliga a definir y mantener dos bloques (spec y body); si la spec cambia, hay que recompilar los programas que la usan.
- Un package siempre se carga completo en memoria, incluso si solo se usa una de sus funciones.
- Si se abusa de elementos privados, la lógica puede volverse difícil de entender para otro desarrollador que solo lee la spec.

Cómo los packages ayudan a organizar el código en el proyecto

En la clínica, los reportes y las validaciones estaban dispersos en objetos separados. El package pkg_reportes_clinica los integra en un solo módulo: sus usuarios solo ven la lista de funciones y procedimientos públicos de la spec, y todas las reglas internas (validar que el médico exista, contar los usos) quedan resueltas dentro del cuerpo.

Relación con los procedimientos y funciones

Un package no reemplaza a los procedimientos ni a las funciones: los agrupa y los relaciona. La función pública fn_citas_medico del package reutiliza internamente a la función privada fn_medico_existe (creada solo para el package) y comparte con el procedimiento sp_mostrar_usos la variable privada v_usos. Así, los objetos individuales y el package trabajan juntos como una solución integral.

Demostración: package con parte pública y privada

La especificación pública solo declara fn_citas_medico y sp_mostrar_usos:

```
CREATE OR REPLACE PACKAGE pkg_reportes_clinica IS

    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER;

    PROCEDURE sp_mostrar_usos;

END pkg_reportes_clinica;
/
```

El cuerpo implementa ambas y define además la parte privada (variable v_usos y función fn_medico_existe), invisible desde afuera:

```
CREATE OR REPLACE PACKAGE BODY pkg_reportes_clinica IS

    -- PARTE PRIVADA 1: variable interna del package (vive por sesion)
    v_usos NUMBER := 0;

    -- PARTE PRIVADA 2: funcion de apoyo, no declarada en la spec
```

```
    FUNCTION fn_medico_existe (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_cuenta NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cuenta
        FROM medico WHERE id_medico = p_id_medico;
        RETURN v_cuenta;
    END fn_medico_existe;

    -- PARTE PUBLICA: funcion declarada en la ejecutacion de la spec
    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        v_usos := v_usos + 1;

        IF fn_medico_existe(p_id_medico) = 0 THEN
            RETURN -1;   -- medico inexistente (regla interna)
        END IF;

        SELECT COUNT(*) INTO v_total
        FROM cita WHERE id_medico = p_id_medico;

        RETURN v_total;
    EXCEPTION
        WHEN OTHERS THEN
            RETURN -2;
    END fn_citas_medico;

    -- PARTE PUBLICA: procedimiento de la spec que expone el contador
    PROCEDURE sp_mostrar_usos IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Este package se ha usado ' || v_usos
            || ' veces en esta sesion.');
    END sp_mostrar_usos;

END pkg_reportes_clinica;
/
```

Las validaciones del package cubren cuatro casos: la función pública aplicada a los 10 médicos, un médico inexistente que devuelve -1, el uso desde un bloque PL/SQL con el contador interno y el intento de acceso a la función privada:

```
-- Validacion 1: uso de la funcion en SQL sobre todos los medicos.
SELECT m.id_medico AS medico,
       m.nombre || ' ' || m.apellido AS nombre_medico,
       pkg_reportes_clinica.fn_citas_medico(m.id_medico) AS citas_atendidas
FROM medico m
ORDER BY m.id_medico;

-- Validacion 2: medico inexistente; devuelve -1 (validacion privada).
SELECT pkg_reportes_clinica.fn_citas_medico(9999) AS medico_inexistente
FROM dual;

-- Validacion 3: uso del package desde un bloque PL/SQL.
DECLARE
    v_citas_medico_2 NUMBER;
BEGIN
    v_citas_medico_2 := pkg_reportes_clinica.fn_citas_medico(2);
    DBMS_OUTPUT.PUT_LINE('El medico 2 tiene ' || v_citas_medico_2 || ' citas.');
    pkg_reportes_clinica.sp_mostrar_usos;
END;
/

-- Validacion 4: intento de acceso a la funcion privada desde fuera del package.
-- Oracle rechaza la llamada con PLS-00231 u ORA-00904.
-- SELECT pkg_reportes_clinica.fn_medico_existe(1) FROM dual;
```

Desarrollo de triggers

Definición y características

Un trigger (disparador) es un bloque PL/SQL que Oracle ejecuta de forma AUTOMÁTICA cuando ocurre un evento sobre una tabla (INSERT, UPDATE o DELETE). El programador no lo llama: se dispara solo. Se distinguen dos niveles:

- Nivel de fila (FOR EACH ROW): se ejecuta una vez POR CADA FILA afectada por la sentencia, y puede leer los datos de cada fila con las referencias :NEW y :OLD.
- Nivel de sentencia (sin FOR EACH ROW): se ejecuta UNA SOLA VEZ por cada sentencia, aunque esa sentencia procese cientos de filas.

Ventajas y desventajas

Ventajas:

- Automatizan reglas del negocio: el desarrollador no puede olvidarse de aplicarla, porque la base la ejecuta sola ante cada evento.
- Permiten auditoría (bitácoras) sin tocar el código de las aplicaciones.
- A nivel de fila, con :NEW/:OLD permiten corregir o complementar los datos justo al momento de la operación.

Desventajas:

- Hacen el comportamiento de la base menos transparente: una inserción aparentemente simple puede disparar lógica oculta.
- Demasiados triggers sobre la misma tabla pueden degradar el rendimiento de las operaciones SQL.
- Un error introducido en un trigger puede bloquear todas las operaciones de escritura sobre la tabla, por eso debe testearse con cuidado.

Propósito de los triggers en el proyecto

Los triggers atienden dos reglas del caso clínico: descontar automáticamente el stock de cada medicamento recetado (control de inventario) y registrar un acceso en una bitácora cada vez que se ingresa información en la agenda de citas (trazabilidad de la información).

Demostración: trigger a nivel de fila (Parcial 1)

El trigger trg_descontar_stock se ejecuta una vez por cada detalle de receta insertado y desconta 1 unidad del medicamento correspondiente:

```
CREATE OR REPLACE TRIGGER trg_descontar_stock
AFTER INSERT ON receta_detalle
FOR EACH ROW
DECLARE
    v_stock_actual medicamento.stock%TYPE;
```

```
BEGIN
    SELECT stock INTO v_stock_actual
    FROM medicamento WHERE id_medicamento = :NEW.id_medicamento;

    IF v_stock_actual > 0 THEN
        UPDATE medicamento
        SET stock = stock - 1
        WHERE id_medicamento = :NEW.id_medicamento;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Aviso: el medicamento '
            || :NEW.id_medicamento || ' ya no tiene stock disponible.');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_descontar_stock: ' || SQLERRM);
END trg_descontar_stock;
/
```

Demostración: trigger a nivel de sentencia (Parcial 2)

El trigger trg_bitacora_cita no declara FOR EACH ROW: se dispara una sola vez por cada sentencia de inserción sobre la tabla cita, aunque esa sentencia inserte varias filas a la vez. Apoya su trabajo en la tabla bitacora_citas:

```
CREATE TABLE bitacora_citas (
    id_bitacora   NUMBER(6)  CONSTRAINT pk_bitacora_citas PRIMARY KEY,
    fecha_evento  DATE       DEFAULT SYSDATE,
    usuario_bd    VARCHAR2(60),
    detalle       VARCHAR2(200)
);

CREATE OR REPLACE TRIGGER trg_bitacora_cita
AFTER INSERT ON cita
DECLARE
    v_nuevo_id NUMBER;
BEGIN
    SELECT NVL(MAX(id_bitacora), 0) + 1 INTO v_nuevo_id FROM bitacora_citas;

    INSERT INTO bitacora_citas (id_bitacora, usuario_bd, detalle)
    VALUES (v_nuevo_id, USER,
        'Se ejecuto una insercion en la tabla CITA (nivel de sentencia)');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_bitacora_cita: ' || SQLERRM);
END trg_bitacora_cita;
/
```

La validación inserta 3 citas nuevas en una sola sentencia INSERT ... SELECT (una inserción masiva). Como el trigger es de sentencia, la bitácora recibe apenas 1 registro, no 3:

```
DECLARE
    v_base NUMBER;
BEGIN
    SELECT NVL(MAX(id_cita), 0) INTO v_base FROM cita;

    INSERT INTO cita (id_cita, id_paciente, id_medico, id_consultorio,
                      fecha_cita, hora_cita, estado)
    SELECT v_base + ROWNUM, id_paciente, 1, 1,
           SYSDATE + 7, '09:00', 'PROGRAMADA'
    FROM paciente
    WHERE ROWNUM <= 3;

    DBMS_OUTPUT.PUT_LINE('La sentencia inserto ' || SQL%ROWCOUNT
        || ' citas de una sola vez.');
    COMMIT;
END;
/

-- Comprobacion: la bitacora gano solo UN registro
SELECT id_bitacora, TO_CHAR(fecha_evento, 'DD-MM-YYYY HH24:MI') AS fecha,
```

```
       usuario_bd, detalle
FROM bitacora_citas
ORDER BY id_bitacora DESC
FETCH FIRST 3 ROWS ONLY;
```

Al comparar ambos triggers queda clara la diferencia: el trigger de fila descuenta el stock una vez por cada fila, mientras que el trigger de sentencia registra una sola vez por sentencia. Cada nivel sirve a una regla distinta: fila para operar con los datos de cada registro, y sentencia para acciones que ocurren una vez por operación.

Conclusión

Resumen

La segunda parte del caso resolutivo completó la capa procedural: se implementaron procedimientos con y sin parámetros (sp_registrar_cita y sp_resumen_clinica) y funciones con y sin parámetros (fn_calcular_edad y fn_total_pacientes), demostrando su uso tanto en sentencias SQL como en otros programas PL/SQL. El package pkg_reportes_clinica integró dichos objetos en un módulo organizado, con una interfaz pública (spec) y elementos privados (body) que encapsulan validación y control interno. Finalmente, los triggers trg_descontar_stock (nivel de fila, Parcial 1) y trg_bitacora_cita (nivel de sentencia, Parcial 2) automatizan las reglas de negocio de stock y trazabilidad de la agenda.

Impacto del Proyecto

La utilización de estos objetos contribuyó directamente a la solución propuesta: los procedimientos centralizan las acciones del negocio (registrar citas, generar el resumen) evitando que cada programa repita código; las funciones permiten reutilizar cálculos como la edad de los 120 pacientes dentro de consultas masivas; el package entrega a los desarrolladores una única puerta de entrada a los reportes, protegiendo la lógica interna; y los triggers garantizan que el stock y la bitácora estén siempre correctos, indistintamente de la aplicación que registre los datos. En conjunto, la base de datos pasa de solo almacenar información a procesarla y generar valor a partir de ella.

Recomendaciones

Como mejoras futuras se propone:

- Reemplazar la generación de ids con NVL(MAX(...))+1 por secuencias (CREATE SEQUENCE), que garantizan llaves únicas de forma segura ante procesos concurrentes.
- Incorporar técnicas BULK COLLECT y FORALL cuando los volúmenes crecieran más allá de los datos actuales, para procesar de forma más eficiente sin recorrer fila a fila.
- Programar la ejecución automática del reporte general mediante DBMS_SCHEDULER, dejando el resumen disponible cada mañana.

- Extender la bitácora a las demás tablas críticas (citas, diagnósticos, tratamientos) y registrar también UPDATE y DELETE.

Anexos

A.1 Código completo

A continuación se incluye el código de los objetos nuevos de la EP2: la tabla de apoyo, el procedimiento sin parámetros, la función sin parámetros, el package público/privado y el trigger a nivel de sentencia, con sus validaciones y la verificación de la parte privada del package (PLS-00231 / ORA-00904). Además, el archivo BD_Clinica_EP2_Completa.sql incluido en la entrega consolida la base de datos de la evaluación anterior y los objetos de esta evaluación en un solo script autoejecutable (12 tablas, carga masiva, ejemplos 1 a 9 y los objetos de la EP2), de modo que el proyecto puede reproducirse desde cero sin depender de archivos externos.

```
-- ========================================================================
-- BDY1103 - TALLER DE BASE DE DATOS
-- CASO DE ANALISIS: SISTEMA DE GESTION DE UNA CLINICA
-- EVALUACION PARCIAL 2: PROCEDIMIENTOS, FUNCIONES, PACKAGES Y TRIGGERS
-- Estudiante: Sebastian Antipan
-- ========================================================================
-- Estos objetos se agregan sobre la base completa que crea el script
-- BD_Clinica_EP2_Completa.sql. La numeracion de ejemplos continua
-- la de la base de datos.
--
--
-- CONTENIDO:
-- 6. Tabla de apoyo bitacora_citas
-- 7. EJEMPLO 10: PROCEDIMIENTO SIN PARAMETROS (sp_resumen_clinica)
-- 8. EJEMPLO 11: FUNCION SIN PARAMETROS (fn_total_pacientes)
-- 9. EJEMPLO 12: PACKAGE CON PARTE PUBLICA Y PRIVADA (pkg_reportes_clinica)
-- 10. EJEMPLO 13: TRIGGER A NIVEL DE SENTENCIA (trg_bitacora_cita)
-- 11. VERIFICACION FINAL DE VOLUMENES
-- ========================================================================

SET SERVEROUTPUT ON SIZE UNLIMITED;



-- ========================================================================
-- 6. TABLA DE APOYO: BITACORA_CITAS
-- ========================================================================
-- Requerida por el trigger a nivel de sentencia (EJEMPLO 13). Guarda
-- un solo registro cada vez que se ejecuta una insercion sobre la
-- tabla CITA, de modo que quede una bitacora de accesos a los datos.
-- ========================================================================


-- Se elimina la tabla de apoyo si ya existe, para permitir una nueva
-- ejecucion completa del script.
DECLARE
BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE bitacora_citas PURGE';
EXCEPTION
    WHEN OTHERS THEN
        NULL;   -- si no existe, se ignora
END;
/

CREATE TABLE bitacora_citas (
```

```
    id_bitacora   NUMBER(6)   CONSTRAINT pk_bitacora_citas PRIMARY KEY,
    fecha_evento  DATE        DEFAULT SYSDATE,
    usuario_bd    VARCHAR2(60),
    detalle       VARCHAR2(200)
);



-- ======================================================================
-- EJEMPLO 10: PROCEDIMIENTO ALMACENADO SIN PARAMETROS
-- ======================================================================
-- Un procedimiento SIN parametros no recibe datos de entrada: trabaja
-- con TODO el conjunto de informacion de la clinica. Aqui genera el
-- resumen general (procesamiento masivo sobre los 120 pacientes, los
-- medicos, las citas y el stock de medicamentos).
-- ======================================================================

CREATE OR REPLACE PROCEDURE sp_resumen_clinica IS

    v_total_pacientes    NUMBER;
    v_total_medicos      NUMBER;
    v_total_citas        NUMBER;
    v_stock_medicamentos NUMBER;

BEGIN
    SELECT COUNT(*) INTO v_total_pacientes FROM paciente;
    SELECT COUNT(*) INTO v_total_medicos   FROM medico;
    SELECT COUNT(*) INTO v_total_citas     FROM cita;
    SELECT NVL(SUM(stock), 0) INTO v_stock_medicamentos FROM medicamento;

    DBMS_OUTPUT.PUT_LINE('===== RESUMEN GENERAL DE LA CLINICA =====');
    DBMS_OUTPUT.PUT_LINE('Pacientes registrados    : ' || v_total_pacientes);
    DBMS_OUTPUT.PUT_LINE('Medicos registrados      : ' || v_total_medicos);
    DBMS_OUTPUT.PUT_LINE('Citas registradas        : ' || v_total_citas);
    DBMS_OUTPUT.PUT_LINE('Unidades de medicamentos : ' || v_stock_medicamentos);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error inesperado: ' || SQLERRM);
END sp_resumen_clinica;
/

-- Validacion 1: invocacion del procedimiento desde una sentencia SQL.
-- La salida se obtiene con DBMS_OUTPUT.
CALL sp_resumen_clinica();

-- Validacion 2: invocacion del procedimiento desde un bloque PL/SQL.
DECLARE
    v_mensaje VARCHAR2(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('Ejecutando el reporte general automaticamente...');
    sp_resumen_clinica;
    v_mensaje := 'Reporte generado el ' || TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI');
    DBMS_OUTPUT.PUT_LINE(v_mensaje);
END;
/



-- ======================================================================
-- EJEMPLO 11: FUNCION ALMACENADA SIN PARAMETROS
-- ======================================================================
-- La funcion SIN parametros tampoco recibe entrada, pero SI devuelve
-- un valor con RETURN (al contrario del procedimiento). Por eso puede
-- usarse directamente en un SELECT.
-- ======================================================================

CREATE OR REPLACE FUNCTION fn_total_pacientes RETURN NUMBER IS
```

```
    v_total NUMBER;

BEGIN
    SELECT COUNT(*) INTO v_total FROM paciente;
    RETURN v_total;

EXCEPTION
    WHEN OTHERS THEN
        RETURN -1;
END fn_total_pacientes;
/

-- Validacion 1: uso de la funcion en una sentencia SQL.
SELECT fn_total_pacientes AS total_pacientes_de_la_clinica FROM dual;

-- Validacion 2: uso de la funcion en un bloque PL/SQL.
DECLARE
    v_total NUMBER;
BEGIN
    v_total := fn_total_pacientes;
    DBMS_OUTPUT.PUT_LINE('La clinica atiende hoy a ' || v_total
        || ' pacientes.');
END;
/

-- Validacion 3: uso de fn_calcular_edad en un bloque PL/SQL.
DECLARE
    v_edad NUMBER;
BEGIN
    v_edad := fn_calcular_edad(1);
    DBMS_OUTPUT.PUT_LINE('La edad del paciente 1 es ' || v_edad
        || ' anios (calculada con fn_calcular_edad).');
END;
/



-- ========================================================================
-- EJEMPLO 12: PACKAGE CON PARTE PUBLICA Y PARTE PRIVADA
-- ========================================================================
-- Un package tiene dos bloques:
--   * ESPECIFICACION (spec): es lo PUBLICO. Declara los elementos que
--     cualquier programa o sentencia SQL puede invocar.
--   * CUERPO (body): implementa la logica y puede incluir elementos
--     PRIVADOS (variables y funciones que aparecen solo aqui, no en la
--     spec). Esos elementos privados son de uso interno del package y
--     nadie de afuera puede llamarlos.
--
-- En este package:
--   PUBLICO : fn_citas_medico (cuenta las citas atendidas de un medico)
--             sp_mostrar_usos (muestra el contador interno)
--   PRIVADO : v_usos (variable interna del package)
--             fn_medico_existe (funcion de apoyo para validar datos)
-- ========================================================================

-- ---------- ESPECIFICACION (lo que se ve desde afuera) ---------------
CREATE OR REPLACE PACKAGE pkg_reportes_clinica IS

    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER;

    PROCEDURE sp_mostrar_usos;

END pkg_reportes_clinica;
/
```

```
-- ---------- CUERPO (lo que se ve y lo que queda privado) --------------
CREATE OR REPLACE PACKAGE BODY pkg_reportes_clinica IS

    -- PARTE PRIVADA 1: variable interna del package. Vive mientras
    -- dure la sesion de trabajo y solo el propio package la puede
    -- leer o modificar.
    v_usos NUMBER := 0;


    -- PARTE PRIVADA 2: funcion de apoyo. NO esta declarada en la spec,
    -- por lo que no puede llamarse desde afuera: solo sirve dentro del
    -- body para validar que el medico exista.
    FUNCTION fn_medico_existe (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_cuenta NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cuenta
        FROM medico
        WHERE id_medico = p_id_medico;


        RETURN v_cuenta;
    END fn_medico_existe;


    -- PARTE PUBLICA: implementa la funcion declarada en la spec.
    FUNCTION fn_citas_medico (
        p_id_medico IN medico.id_medico%TYPE
    ) RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        v_usos := v_usos + 1;


        IF fn_medico_existe(p_id_medico) = 0 THEN
            RETURN -1;   -- medico inexistente (regla interna)
        END IF;


        SELECT COUNT(*) INTO v_total
        FROM cita
        WHERE id_medico = p_id_medico;


        RETURN v_total;


    EXCEPTION
        WHEN OTHERS THEN
            RETURN -2;
    END fn_citas_medico;


    -- PARTE PUBLICA: expone el contador privado (unico acceso externo
    -- a la variable interna).
    PROCEDURE sp_mostrar_usos IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Este package se ha usado ' || v_usos
            || ' veces en esta sesion.');
    END sp_mostrar_usos;


END pkg_reportes_clinica;
/


-- Validacion 1: uso de la funcion publica del package en una sentencia
-- SQL sobre todos los medicos. Devuelve la cantidad de citas de cada uno.
SELECT m.id_medico AS medico,
       m.nombre || ' ' || m.apellido AS nombre_medico,
       pkg_reportes_clinica.fn_citas_medico(m.id_medico) AS citas_atendidas
FROM medico m
ORDER BY m.id_medico;
```

```
-- Validacion 2: medico inexistente. La parte privada valida los datos
-- y devuelve -1.
SELECT pkg_reportes_clinica.fn_citas_medico(9999) AS medico_inexistente
FROM dual;


-- Validacion 3: uso del package desde un bloque PL/SQL, consultando el
-- contador interno por medio del procedimiento publico.
DECLARE
    v_citas_medico_2 NUMBER;
BEGIN
    v_citas_medico_2 := pkg_reportes_clinica.fn_citas_medico(2);
    DBMS_OUTPUT.PUT_LINE('El medico 2 tiene ' || v_citas_medico_2
        || ' citas registradas.');
    pkg_reportes_clinica.sp_mostrar_usos;
END;
/


-- Validacion 4: intento de acceso a fn_medico_existe desde fuera del
-- package. Oracle rechaza la llamada con PLS-00231 u ORA-00904; el
-- error se controla con SQL dinamico para continuar con el script.
DECLARE
    v_resultado NUMBER;
BEGIN
    EXECUTE IMMEDIATE
        'SELECT pkg_reportes_clinica.fn_medico_existe(1) FROM dual'
        INTO v_resultado;
    DBMS_OUTPUT.PUT_LINE('Resultado inesperado: ' || v_resultado);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error esperado al llamar la parte privada: '
            || SQLERRM);
END;
/



-- ========================================================================
-- EJEMPLO 13: TRIGGER A NIVEL DE SENTENCIA
-- ========================================================================
-- El trigger del Parcial 1 (EJEMPLO 9) era "FOR EACH ROW": se dispara
-- una vez POR CADA FILA insertada. Este trigger NO tiene FOR EACH ROW,
-- por lo que se dispara UNA SOLA VEZ por cada sentencia INSERT que se
-- ejecute, aunque esa sentencia inserte 1 fila o 500 filas a la vez.
-- Cada disparo queda registrado en la bitacora de la seccion 6.
-- ========================================================================


CREATE OR REPLACE TRIGGER trg_bitacora_cita
AFTER INSERT ON cita
DECLARE
    v_nuevo_id NUMBER;
BEGIN
    SELECT NVL(MAX(id_bitacora), 0) + 1 INTO v_nuevo_id FROM bitacora_citas;


    INSERT INTO bitacora_citas (
        id_bitacora, usuario_bd, detalle
    ) VALUES (
        v_nuevo_id,
        USER,
        'Se ejecuto una insercion en la tabla CITA '
        || '(nivel de sentencia)'
    );


EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error en trigger trg_bitacora_cita: ' || SQLERRM);
END trg_bitacora_cita;
/


-- Validacion: insercion de 3 citas en UNA SOLA sentencia. El trigger
-- de sentencia se dispara una sola vez, por lo que la bitacora registra
```

```
-- 1 fila y no 3. Asi se diferencia del trigger de fila.
DECLARE
    v_base NUMBER;
BEGIN
    SELECT NVL(MAX(id_cita), 0) INTO v_base FROM cita;


    INSERT INTO cita (
        id_cita, id_paciente, id_medico, id_consultorio,
        fecha_cita, hora_cita, estado
    )
    SELECT v_base + ROWNUM,
           id_paciente,
           1,                       -- medico 1
           1,                       -- consultorio 1
           SYSDATE + 7,             -- control dentro de una semana
           '09:00',
           'PROGRAMADA'
    FROM paciente
    WHERE ROWNUM <= 3;


    DBMS_OUTPUT.PUT_LINE('La sentencia inserto ' || SQL%ROWCOUNT
        || ' citas de una sola vez.');
    COMMIT;
END;
/


-- Consulta de la bitacora. Se espera un unico registro nuevo,
-- correspondiente a las 3 citas insertadas en una sola sentencia.
SELECT id_bitacora, TO_CHAR(fecha_evento, 'DD-MM-YYYY HH24:MI') AS fecha,
       usuario_bd, detalle
FROM bitacora_citas
ORDER BY id_bitacora DESC
FETCH FIRST 3 ROWS ONLY;


-- Total de registros en la bitacora.
SELECT COUNT(*) AS total_bitacora FROM bitacora_citas;



-- ================================================================
-- 11. VERIFICACION FINAL DE VOLUMENES
-- ================================================================
-- Consulta de cierre: muestra los totales de las tablas principales
-- despues de ejecutar los objetos y sus validaciones.
-- ================================================================


DECLARE
    v_pacientes   NUMBER;
    v_medicos     NUMBER;
    v_citas       NUMBER;
    v_stock       NUMBER;
    v_bitacora    NUMBER;
    v_recetas     NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_pacientes FROM paciente;
    SELECT COUNT(*) INTO v_medicos   FROM medico;
    SELECT COUNT(*) INTO v_citas     FROM cita;
    SELECT NVL(SUM(stock), 0) INTO v_stock FROM medicamento;
    SELECT COUNT(*) INTO v_bitacora  FROM bitacora_citas;
    SELECT COUNT(*) INTO v_recetas   FROM receta_detalle;


    DBMS_OUTPUT.PUT_LINE('============================================');
    DBMS_OUTPUT.PUT_LINE(' RESUMEN FINAL DE VOLUMENES');
    DBMS_OUTPUT.PUT_LINE('============================================');
    DBMS_OUTPUT.PUT_LINE('Pacientes                : ' || v_pacientes);
    DBMS_OUTPUT.PUT_LINE('Medicos                  : ' || v_medicos);
    DBMS_OUTPUT.PUT_LINE('Citas (estado final)     : ' || v_citas);
    DBMS_OUTPUT.PUT_LINE('Stock total de medicamentos: ' || v_stock);
    DBMS_OUTPUT.PUT_LINE('Detalles de receta       : ' || v_recetas);
    DBMS_OUTPUT.PUT_LINE('Registros en bitacora    : ' || v_bitacora);
```

```
    DBMS_OUTPUT.PUT_LINE('============================================');
END;
/
```

A.2 Diagramas y modelos

Modelo de datos (12 tablas, Parcial 1): las entidades principales son paciente, medico y consultorio; cita relaciona las tres; cada cita genera un diagnostico, que puede originar un tratamiento y/o una receta; la receta enlaza con medicamento a través de receta_detalle; historial_medico guarda la ficha general del paciente; y el catálogo lo completan especialidad, medico_especialidad y medicamento. La tabla bitacora_citas (Parcial 2) se agrega como apoyo del trigger a nivel de sentencia. En la Figura 1 se presenta el diagrama completo del modelo.

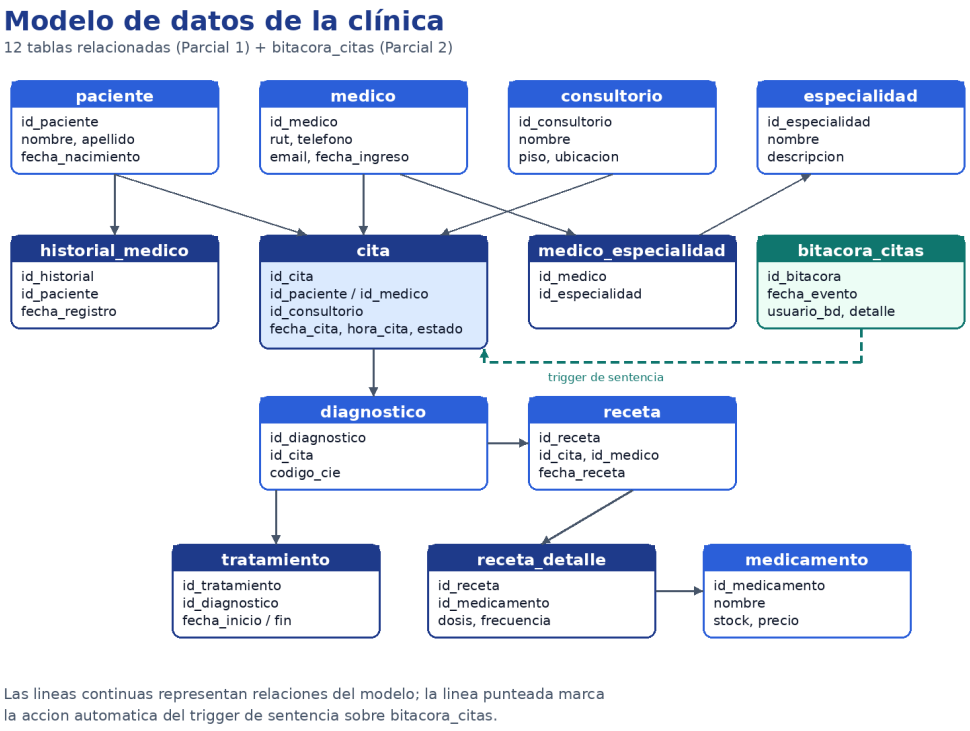


Figura 1. Modelo de datos de la clínica (12 tablas y bitacora_citas).

Mapa de los objetos PL/SQL del proyecto. La Figura 2 resume visualmente cómo se relacionan los objetos con las tablas y con las acciones de la clínica:

```
sp_registrar_cita  (PROC con parametros) -> INSERT en cita
sp_resumen_clinica (PROC sin parametros) -> 4 SELECT COUNT masivos
fn_calcular_edad   (FUNC con parametros) -> SELECT, bloques PL/SQL
fn_total_pacientes (FUNC sin parametros) -> SELECT ... FROM dual
pkg_reportes_clinica (PACKAGE)
  |- parte PUBLICA : fn_citas_medico, sp_mostrar_usos
  |- parte PRIVADA : v_usos, fn_medico_existe
trg_descontar_stock  (TRIGGER fila)  : receta_detalle -> medicamento
trg_bitacora_cita    (TRIGGER sentencia) : cita -> bitacora_citas
```

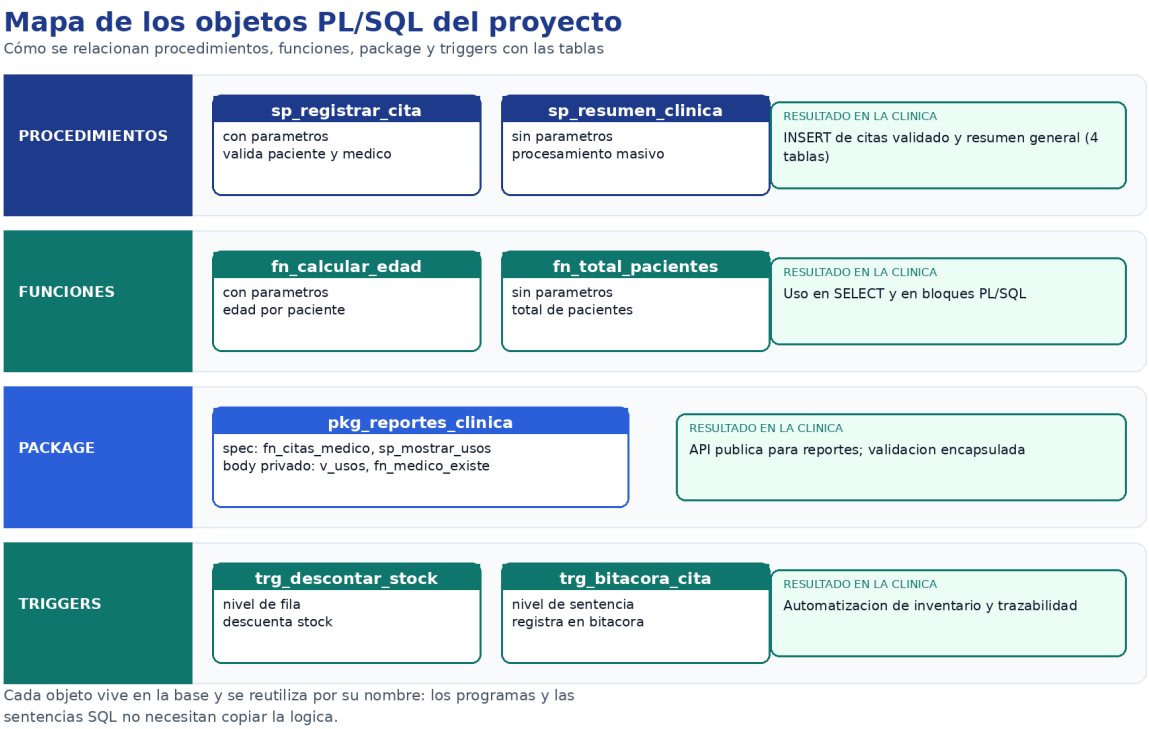


Figura 2. Mapa de los objetos PL/SQL y su relación con las tablas y acciones de la clínica.

A.3 Resultados de la ejecución

La ejecución completa se realizó en FreeSQL sobre la base de la EP1. Los valores obtenidos son los siguientes:

Objeto / consulta	Resultado real	Observación
sp_resumen_clinica	120 pacientes, 10 médicos, 243 citas y 3489 unidades de stock	Se ejecutó antes de la prueba del trigger.
fn_total_pacientes	120	Función sin parámetros.
fn_calcular_edad	18, 48, 23, 36, 46, 66, 49, 44, 49 y 60 años (10 primeros pacientes)	Función con parámetros, usada en SQL y en PL/SQL.
pkg_reportes_clinica	10 médicos: 29, 22, 28, 24, 25, 14, 26, 20, 22 y 33 citas; 9999 = -1; contador = 12 usos	Corte antes de la prueba del trigger; la función privada no es accesible desde fuera.
trg_bitacora_cita	3 citas en una sentencia; 1 registro en la bitácora (27-10-2026 16:55)	Trigger de sentencia.
Estado final	246 citas, 3489 unidades de stock, 113 detalles de receta y 1 registro en la bitácora	Después de la prueba del trigger.

La llamada a la función privada se rechazó con ORA-00904 (equivalente a PLS-00231 en una llamada PL/SQL directa), lo que confirma que el encapsulamiento del package funciona.